

[SUDO] pcscd

DESCRIPTION : Daemon PC/SC – couche d'abstraction entre le système et les lecteurs de cartes à puce (smartcard readers)

POURQUOI : pcscd est le prérequis indispensable à tout l'écosystème smartcard sous Linux. Il gère la communication avec les lecteurs via le standard PC/SC et expose une API aux applications (OpenSC, gpg-agent, Firefox...).

CONTEXTE : Doit être démarré avant toute utilisation de carte à puce.

OPTIONS :

- f Mode foreground (ne passe pas en daemon)
- d Mode débogage
- a Mode auto-exit (s'arrête si aucun lecteur)
- disable-polkit Désactive l'authentification PolicyKit

EXEMPLES :

```
$ sudo systemctl start pcscd
$ sudo systemctl enable pcscd
$ sudo pcscd -f -d
$ sudo systemctl status pcscd
```

[USER] opensc-tool

DESCRIPTION : Outil générique d'interaction avec les smartcards via OpenSC

POURQUOI : Accéder directement à une carte à puce – lister les lecteurs, identifier la carte insérée, envoyer des commandes APDU brutes, afficher les informations de base de la carte.

CONTEXTE : Diagnostic, identification de cartes, développement, tests.

OPTIONS :

- l / --list-readers Liste tous les lecteurs de cartes détectés
- n / --name Affiche le nom de la carte insérée
- a / --atr Affiche l'ATR (Answer To Reset) de la carte
- r LECTEUR Utilise ce lecteur (nom ou numéro)
- s APDU Envoie cette commande APDU à la carte
- f FICHIER Lit les commandes APDU depuis un fichier
- wait Attend qu'une carte soit insérée
- verbose / -v Mode verbeux (peut être répété)

ATR (Answer To Reset) :

Séquence d'octets émise par la carte à l'insertion – identifie le type de carte, le protocole (T=0, T=1), la vitesse de communication.

EXEMPLES :

```
$ opensc-tool --list-readers
$ opensc-tool --atr
$ opensc-tool --name
$ opensc-tool -s "00 A4 04 00"
$ opensc-tool -r "Yubico YubiKey" --atr
$ opensc-tool --wait --atr
```

[USER] opensc-explorer

DESCRIPTION : Shell interactif d'exploration de la structure d'une smartcard – navigue dans les fichiers EF/DF de la carte

POURQUOI : Explorer manuellement l'arborescence d'une carte à puce (structure ISO 7816) – lire des fichiers élémentaires (EF), naviguer dans les répertoires dédiés (DF), inspecter les données brutes.

CONTEXTE : Développement smartcard, forensique, audit de cartes, reverse engineering.

OPTIONS :

- r LECTEUR Utilise ce lecteur
- wait Attend l'insertion d'une carte
- verbose Mode verbeux

COMMANDES INTERACTIVES :

- ls Liste les fichiers et répertoires du niveau courant
- cd DF_ID Entre dans un répertoire dédié (DF)
- cat EF_ID Lit et affiche le contenu d'un fichier élémentaire
- info Informations sur l'objet courant
- get EF_ID FICHIER Exporte un EF vers un fichier local
- put EF_ID FICHIER Importe un fichier local dans un EF
- pin Gestion des PIN
- quit Quitte

```

EXEMPLES      :
$ opensc-explorer
$ opensc-explorer --wait
OpenSC Explorer> ls
OpenSC Explorer> cd 3F00
OpenSC Explorer> ls
OpenSC Explorer> cat 2F00

```

```
[USER] opensc-asn1
```

```

DESCRIPTION   : Décode et affiche des structures ASN.1 (format d'encodage des données
                  cryptographiques)
POURQUOI      : Les smartcards stockent leurs données (certificats, clés, structures
                  PKCS#15) en ASN.1/DER – opensc-asn1 les rend lisibles.
CONTEXTE      : Développement smartcard, débogage, analyse de certificats et structures
                  cryptographiques.
OPTIONS       :
-f FICHIER          Décode ce fichier ASN.1/DER
-c                  Sortie compacte
EXEMPLES       :
$ opensc-asn1 -f certificat.der
$ opensc-tool -s "00 B0 00 00 00" | opensc-asn1

```

```
[USER] pkcs15-tool
```

```

DESCRIPTION   : Outil principal d'accès aux objets PKCS#15 stockés sur une smartcard
                  (certificats, clés privées, données d'authentification)
POURQUOI      : PKCS#15 est le standard de structure de données pour les smartcards
                  cryptographiques. pkcs15-tool permet de lister, lire et exporter les
                  objets cryptographiques (certificats X.509, clés publiques, PINs...).
CONTEXTE      : Gestion de smartcards PKI, export de certificats, audit de cartes,
                  authentification.
OPTIONS       :
-D / --dump          Affiche tous les objets PKCS#15 de la carte
-c / --list-certificates  Liste les certificats
-k / --list-keys      Liste les clés privées
-p / --list-public-keys  Liste les clés publiques
-d / --list-data-objects  Liste les objets de données
-P / --list-pins      Liste les objets PIN
-r N / --read-certificate=N  Lit et affiche le certificat N
-R N / --read-public-key=N  Lit et affiche la clé publique N
--read-data-object=LABEL  Lit un objet de données par son label
-o FICHIER           Fichier de sortie
-f FORMAT            Format de sortie (PEM, DER)
--verify-pin         Vérifie le PIN
--change-pin         Change le PIN
--unblock-pin        Débloque le PIN
-w / --wait          Attend l'insertion d'une carte
-r LECTEUR           Utilise ce lecteur
-v                  Mode verbeux
EXEMPLES       :
$ pkcs15-tool --dump
$ pkcs15-tool --list-certificates
$ pkcs15-tool --list-keys
$ pkcs15-tool --read-certificate 0 -o certificat.pem -f PEM
$ pkcs15-tool --read-public-key 0
$ pkcs15-tool --verify-pin
$ pkcs15-tool --change-pin
$ pkcs15-tool --list-pins

```

```
[USER] pkcs15-init
```

```

DESCRIPTION   : Initialise une smartcard vierge avec une structure PKCS#15 et génère
                  ou importe des clés et certificats
POURQUOI      : Personnaliser une carte à puce – créer une PKI sur carte, importer
                  des certificats et clés privées, configurer les PINs.
CONTEXTE      : Déploiement de smartcards PKI, administration de cartes, laboratoires.
AVERTISSEMENT : Efface et reconfigure la carte – toutes les données existantes
                  seront perdues.
OPTIONS       :
-E / --erase-card    Efface complètement la carte
-C / --create-pkcs15  Crée une structure PKCS#15

```

-p PROFIL Profil à utiliser (pkcs15, muscle, asepcos...)
 -G / --generate-key ALG/TAILLE Génère une paire de clés sur la carte
 -S / --store-private-key FICH Importe une clé privée
 -P / --store-public-key FICH Importe une clé publique
 -X / --store-certificate FICH Importe un certificat
 --store-data FICHIER Importe des données arbitraires
 -a / --auth-id ID ID d'authentification requis pour l'objet
 --pin PIN PIN utilisateur
 --puk PUK PUK (PIN de déblocage)
 --so-pin PIN PIN de l'administrateur (Security Officer)
 -i ID ID de l'objet à créer
 -l LABEL Label de l'objet
 -u UTILISATION Utilisation de la clé (sign, decrypt, encrypt...)
 -r LECTEUR Utilise ce lecteur

EXEMPLES :

```

$ pkcs15-init --erase-card
$ pkcs15-init --create-pkcs15 --profile pkcs15 --pin 1234 --puk 12345678
$ pkcs15-init --generate-key rsa/2048 --auth-id 01 --label "Ma Clé RSA"
$ pkcs15-init --store-certificate cert.pem --label "Mon Certificat"
$ pkcs15-init --store-private-key cle.pl2 --format PKCS12 --label "Ma Clé"

```

[USER] pkcs15-crypt

DESCRIPTION : Effectue des opérations cryptographiques (signature, déchiffrement) en utilisant les clés stockées sur la smartcard
 POURQUOI : Signer ou déchiffrer des données avec une clé privée qui ne quitte jamais la carte – la clé reste protégée matériellement.
 CONTEXTE : Signature numérique, déchiffrement sécurisé, authentification.
 OPTIONS :
 -s / --sign Mode signature
 -d / --decipher Mode déchiffrement
 -v / --verify Mode vérification de signature
 -k N / --key=N Numéro de la clé à utiliser
 -i FICHIER Fichier d'entrée (données à signer/déchiffrer)
 -o FICHIER Fichier de sortie
 --sha-1 / --sha-256 Algorithme de hash pour la signature
 --pkcs1 Utilise le padding PKCS#1
 --pin PIN PIN pour accéder à la clé
 -r LECTEUR Utilise ce lecteur

EXEMPLES :

```

$ pkcs15-crypt --sign -k 1 -i document.txt -o signature.bin
$ pkcs15-crypt --decipher -k 1 -i message_chiffre.bin -o message_clair.txt
$ pkcs15-crypt --sign --sha-256 --pkcs1 -i fichier.txt -o sig.bin

```

[USER] pkcs11-tool

DESCRIPTION : Outil d'accès aux tokens PKCS#11 – interface standardisée pour les smartcards et HSM (Hardware Security Modules)
 POURQUOI : PKCS#11 est l'API standard pour accéder aux tokens cryptographiques matériels. pkcs11-tool permet de gérer les objets, tester les mécanismes et effectuer des opérations cryptographiques via n'importe quel module PKCS#11 (OpenSC, SoftHSM, YubiKey, etc.).
 CONTEXTE : Gestion de tokens PKI, HSM, YubiKey, smartcards, tests de modules PKCS#11.
 OPTIONS :
 -m MODULE Chemin vers le module PKCS#11 (.so)
 -L / --list-slots Liste les slots disponibles
 -T / --list-token-slots Liste les slots avec token
 -O / --list-objects Liste les objets du token
 -I / --list-mechanisms Liste les mécanismes supportés
 --test Lance les tests de base du token
 --slot N Utilise ce slot
 --token-label LABEL Sélectionne le token par son label
 --login Se connecte avec le PIN utilisateur
 --pin PIN PIN utilisateur
 --so-pin PIN PIN administrateur (Security Officer)
 --init-token Initialise le token
 --label LABEL Label du token
 --init-pin Initialise le PIN utilisateur
 --change-pin Change le PIN
 --generate-random N Génère N octets aléatoires
 --generate-keypair Génère une paire de clés
 --key-type TYPE Type de clé (RSA:2048, EC:prime256v1...)

```

--id ID                ID de l'objet (hex)
--label LABEL          Label de l'objet
--read-object          Lit un objet (certificat, clé publique...)
--write-object FICHIER  Écrit un objet sur le token
--delete-object        Supprime un objet
--type TYPE            Type d'objet (cert, pubkey, privkey, data)
--sign                Signe des données
--verify              Vérifie une signature
--encrypt             Chiffre des données
--decrypt            Déchiffre des données
--input-file FICHIER   Fichier d'entrée
--output-file FICHIER  Fichier de sortie
--mechanism MECA       Mécanisme cryptographique
-v                    Mode verbeux

```

```

MODULES PKCS#11 COURANTS :
/usr/lib64/opensc-pkcs11.so  OpenSC (smartcards génériques)
/usr/lib64/pkcs11/opensc-pkcs11.so
libsofthsm2.so              SoftHSM (HSM logiciel)
/usr/lib/x86_64-linux-gnu/libykcs11.so  YubiKey

```

```

EXEMPLES :
$ pkcs11-tool --list-slots
$ pkcs11-tool -m /usr/lib64/opensc-pkcs11.so --list-token-slots
$ pkcs11-tool --list-objects --login --pin 1234
$ pkcs11-tool --list-mechanisms
$ pkcs11-tool --generate-keypair --key-type RSA:2048 --id 01 \
  --label "Ma Clé" --login --pin 1234
$ pkcs11-tool --read-object --type cert --id 01 -o cert.der
$ pkcs11-tool --generate-random 32 | xxd
$ pkcs11-tool --test --login --pin 1234

```

```
[USER] piv-tool
```

```

DESCRIPTION : Outil dédié aux cartes PIV (Personal Identity Verification –
               standard NIST SP 800-73) utilisées par le gouvernement américain
               et les entreprises (CAC, PIV, YubiKey PIV...)
POURQUOI    : Gérer spécifiquement les cartes PIV – générer des clés dans les
               slots PIV, importer des certificats, lire les données PIV standardisées
               (CHUID, fingerprint...).
CONTEXTE    : Cartes PIV gouvernementales, CAC militaires, YubiKey en mode PIV,
               smartcards d'entreprise compatibles PIV.

```

```

SLOTS PIV STANDARD :
9A                Authentification (Authentication)
9C                Signature numérique (Digital Signature)
9D                Gestion de clés (Key Management)
9E                Authentification de carte (Card Authentication)
82-95            Slots secondaires (Retired Key Management)

```

```

OPTIONS      :
-A ALGO          Algorithme (RSA1024, RSA2048, ECCP256, ECCP384)
-s SLOT          Slot PIV (9A, 9C, 9D, 9E...)
-a / --admin     Authentification administrative
--key CLÉ        Clé de gestion (hex, 24 octets pour 3DES)
-G / --generate  Génère une clé dans le slot spécifié
-S / --cert-req  Génère une demande de certificat (CSR)
-C / --cert-import FICHIER Importe un certificat dans un slot
--set-chuid      Génère et écrit un nouveau CHUID
--set-ccc        Génère et écrit un nouveau CCC
-r LECTEUR       Utilise ce lecteur
-v              Mode verbeux

```

```

EXEMPLES :
$ piv-tool --list-readers
$ piv-tool -G -A RSA2048 -s 9A
$ piv-tool -C cert.pem -s 9A
$ piv-tool -S -s 9A -o demande.csr
$ piv-tool --set-chuid -a --key 010203040506070801020304050607080102030405060708

```

```
[USER] egk-tool
```

```

DESCRIPTION : Outil de lecture des cartes de santé électroniques allemandes
               (elektronische Gesundheitskarte – eGK / carte vitale allemande)
POURQUOI    : Lire et décoder les données stockées sur les cartes de santé
               électroniques allemandes – données administratives, informations
               d'assurance, données médicales d'urgence (NFDm).
CONTEXTE    : Secteur médical allemand, interopérabilité santé, développement

```

d'applications médicales.

```
OPTIONS      :
-r LECTEUR    Utilise ce lecteur
--pd          Lit les données personnelles (Persönliche Daten)
--vd          Lit les données d'assurance (Versicherungsdaten)
--gvd         Lit les données d'assurance améliorées
--nfdm        Lit les données médicales d'urgence (Notfalldaten)
-v           Mode verbeux

EXEMPLES     :
$ egk-tool
$ egk-tool --pd
$ egk-tool --vd
$ egk-tool -r "Reiner SCT" --pd
```

```
[USER] iasecc-tool
```

```
DESCRIPTION  : Outil pour les cartes IAS-ECC (Identification, Authentication,
                Signature – European Citizen Card) – cartes d'identité électroniques
                européennes
POURQUOI     : Accéder aux fonctionnalités des cartes d'identité électroniques
                basées sur le standard IAS-ECC (carte nationale d'identité française,
                belge, etc.).
CONTEXTE     : Cartes nationales d'identité électroniques européennes, e-gouvernement.
OPTIONS      :
-D / --dump   Affiche tous les objets de la carte
-r LECTEUR    Utilise ce lecteur
-v           Mode verbeux
EXEMPLES     :
$ iasecc-tool --dump
$ iasecc-tool -r "ACS ACR122" --dump
```

```
[USER] cardos-tool
```

```
DESCRIPTION  : Outil pour les cartes CardOS (Siemens/Atos) – famille de cartes
                à puce utilisées dans les entreprises et administrations européennes
POURQUOI     : Gérer les cartes CardOS – afficher les informations système, gérer
                le cycle de vie, vérifier l'état de la carte.
CONTEXTE     : Smartcards d'entreprise Siemens/Atos, administrations européennes.
OPTIONS      :
--info        Affiche les informations système de la carte
--startkey N  Utilise la start key N pour l'initialisation
-r LECTEUR    Utilise ce lecteur
-v           Mode verbeux
EXEMPLES     :
$ cardos-tool --info
$ cardos-tool -r "Lecteur 0" --info
```

```
[USER] cryptoflex-tool
```

```
DESCRIPTION  : Outil pour les cartes Schlumberger/Axalto CryptoFlex – une des
                premières smartcards cryptographiques largement déployées
POURQUOI     : Gérer les vieilles cartes CryptoFlex encore présentes dans certains
                environnements legacy.
CONTEXTE     : Systèmes legacy, compatibilité avec des déploiements anciens.
OPTIONS      :
--list-keys   Liste les clés stockées
--generate-key N Génère une clé RSA de N bits
--read-key N  Lit la clé publique N
--store-key FICHER Importe une clé
--verify-pin  Vérifie le PIN
-r LECTEUR    Utilise ce lecteur
EXEMPLES     :
$ cryptoflex-tool --list-keys
$ cryptoflex-tool --generate-key 1024
```

```
[USER] sc-hsm-tool
```

```
DESCRIPTION  : Outil pour le SmartCard-HSM – HSM sur carte à puce open source
                (Nitrokey HSM, SmartCard-HSM)
POURQUOI     : Gérer le SmartCard-HSM – initialiser le device, gérer les clés,
                configurer la politique de backup et les relations de confiance entre
```

```

        plusieurs tokens (DKEK – Device Key Encryption Key).
CONTEXTE      : HSM embarqué, Nitrokey HSM, sécurité haut niveau sur token physique.
OPTIONS       :
--initialize           Initialise le SmartCard-HSM
--so-pin PIN           PIN administrateur
--pin PIN              PIN utilisateur
--label LABEL          Label du token
--dkek-shares N        Nombre de parts DKEK (backup distribué)
--create-dkek-share FICHER Crée une part DKEK
--import-dkek-share FICHER Importe une part DKEK
--wrap-key N           Exporte la clé N (chiffrée avec DKEK)
--unwrap-key N         Importe une clé enveloppée
--key-reference N      Référence de la clé
-r LECTEUR            Utilise ce lecteur
-v                    Mode verbeux
EXEMPLES        :
$ sc-hsm-tool --initialize --so-pin 3537363231383830 --pin 648219 \
  --dkek-shares 2 --label "MonHSM"
$ sc-hsm-tool --create-dkek-share dkek-share-1.pbe
$ sc-hsm-tool --import-dkek-share dkek-share-1.pbe
$ sc-hsm-tool --wrap-key 1 --key-reference 1

```

```
[USER] netkey-tool
```

```

DESCRIPTION    : Outil pour les cartes NetKey (Deutsche Telekom TeleSec) – cartes
                  à puce utilisées en Allemagne pour la signature électronique
POURQUOI       : Gérer les cartes NetKey – changer les PINs, lire les certificats,
                  gérer les clés de signature.
CONTEXTE       : Signature électronique qualifiée allemande, Deutsche Telekom TeleSec.
OPTIONS        :
--list-certs      Liste les certificats
--read-cert N     Lit le certificat N
--verify-pin      Vérifie le PIN
--change-pin      Change le PIN
--unblock-pin     Débloque le PIN
-r LECTEUR        Utilise ce lecteur
EXEMPLES        :
$ netkey-tool --list-certs
$ netkey-tool --read-cert 0
$ netkey-tool --change-pin

```

```
[USER] gids-tool
```

```

DESCRIPTION    : Outil pour les cartes GIDS (Generic Identity Device Specification) –
                  standard Microsoft pour les smartcards sous Windows, supporté par
                  OpenSC sous Linux
POURQUOI       : Initialiser et gérer les cartes GIDS – compatibles nativement avec
                  Windows sans pilote tiers, utilisables sous Linux via OpenSC.
CONTEXTE       : Smartcards d'entreprise compatibles Windows/Linux, déploiements mixtes.
OPTIONS        :
--initialize      Initialise une carte GIDS vierge
--admin-key CLÉ   Clé administrative (hex)
--pin PIN         PIN utilisateur
--label LABEL     Label du token
-r LECTEUR        Utilise ce lecteur
-v               Mode verbeux
EXEMPLES        :
$ gids-tool --initialize --pin 1234 \
  --admin-key 0000000000000000000000000000000000000000000000000000000000000000 \
  --label "MonToken"

```

```
[USER] westcos-tool
```

```

DESCRIPTION    : Outil pour les cartes WestCOS – cartes à puce françaises utilisées
                  notamment pour la carte de professionnel de santé (CPS)
POURQUOI       : Gérer les cartes WestCOS françaises – changer les PINs, lire les
                  informations de la carte.
CONTEXTE       : Cartes de professionnel de santé (CPS) françaises, secteur médical.
OPTIONS        :
--change-pin      Change le PIN
--unblock-pin     Débloque le PIN avec le PUK
-r LECTEUR        Utilise ce lecteur

```

```
EXEMPLES      :  
$ westcos-tool --change-pin  
$ westcos-tool --unblock-pin
```

```
[USER] dtrust-tool
```

```
DESCRIPTION   : Outil pour les cartes D-Trust – cartes de signature électronique  
                qualifiée allemandes (Bundesdruckerei)  
POURQUOI      : Gérer les cartes D-Trust – lire les certificats, gérer les PINs,  
                effectuer des opérations de signature qualifiée.  
CONTEXTE      : Signature électronique qualifiée allemande, Bundesdruckerei.  
OPTIONS       :  
  -r LECTEUR      Utilise ce lecteur  
  -v              Mode verbeux  
EXEMPLES      :  
$ dtrust-tool  
$ dtrust-tool -v
```

```
[USER] eidenv
```

```
DESCRIPTION   : Lit et affiche les données d'une carte d'identité électronique  
                belge (eID) ou compatible  
POURQUOI      : Extraire les informations d'identité (nom, prénom, numéro national,  
                adresse, photo) d'une carte eID belge.  
CONTEXTE      : Carte d'identité électronique belge, authentification eID.  
OPTIONS       :  
  -r LECTEUR      Utilise ce lecteur  
  -t              Format tabulaire  
  -v              Mode verbeux  
  --all           Affiche toutes les données disponibles  
EXEMPLES      :  
$ eidenv  
$ eidenv --all  
$ eidenv -r "ACS ACR38" -t
```

```
[USER] openpgp-tool
```

```
DESCRIPTION   : Outil pour les cartes OpenPGP (Gnuk, YubiKey OpenPGP, Nitrokey...)  
                – tokens de clés PGP matériels  
POURQUOI      : Gérer les tokens OpenPGP matériels – afficher les informations du  
                token, changer les PINs, lire les données du cardholder, vérifier  
                les capacités du token. Complément à gpg --card-edit.  
CONTEXTE      : YubiKey en mode OpenPGP, Nitrokey Pro, Gnuk (FST-01), tokens PGP.  
OPTIONS       :  
  -D / --dump      Affiche toutes les données du token OpenPGP  
  --serial         Affiche le numéro de série  
  --do N           Lit l'objet de données N (Data Object)  
  --verify-pin     Vérifie le PIN  
  --change-pin     Change le PIN  
  --login USER    Se connecte avec cet utilisateur  
  -r LECTEUR       Utilise ce lecteur  
  -v              Mode verbeux  
EXEMPLES      :  
$ openpgp-tool --dump  
$ openpgp-tool --serial  
$ openpgp-tool --verify-pin
```

```
[USER] dnies-tool
```

```
DESCRIPTION   : Outil pour le DNI electrónico – carte d'identité électronique  
                espagnole  
POURQUOI      : Accéder aux données et fonctionnalités cryptographiques du DNIE  
                espagnol – authentification, signature, lecture des données d'identité.  
CONTEXTE      : Carte d'identité électronique espagnole (DNI 3.0), e-gouvernement  
                espagnol.  
OPTIONS       :  
  -r LECTEUR      Utilise ce lecteur  
  -v              Mode verbeux  
EXEMPLES      :  
$ dnies-tool  
$ dnies-tool -v
```

```
[USER] gpg --card-status / gpg --card-edit
```

DESCRIPTION : Interface GPG pour les tokens OpenPGP matériels (YubiKey, Nitrokey, GnuK...) – intégrée dans GnuPG

POURQUOI : Gérer les clés PGP stockées sur un token matériel directement depuis GPG – plus simple que openpgp-tool pour les usages courants.

CONTEXTE : YubiKey, Nitrokey Pro/Start, GnuK, cartes OpenPGP v2/v3.

COMMANDES gpg --card-edit :

admin	Active les commandes administrateur
generate	Génère des clés directement sur le token
passwd	Change les PINs (PIN utilisateur, Admin PIN, PIN reset)
fetch	Récupère la clé publique depuis l'URL configurée
url	Configure l'URL de la clé publique
name	Configure le nom du titulaire
lang	Configure la langue
login	Configure le login
sex	Configure le genre
cafrpr	Configure les fingerprints CA
forcesig	Active la signature forcée sans PIN
quit	Quitte

EXEMPLES :

```
$ gpg --card-status
$ gpg --card-edit
gpg/card> admin
gpg/card> generate
gpg/card> passwd
$ gpg --edit-key KEYID
gpg> keytocard
```

```
[USER] p11-kit
```

DESCRIPTION : Gestionnaire de modules PKCS#11 – couche d'abstraction unifiant l'accès à tous les tokens PKCS#11 du système

POURQUOI : p11-kit centralise la gestion des modules PKCS#11 – évite de spécifier manuellement le chemin du module dans chaque application. Intègre les modules dans le système de confiance (trust store) Linux.

CONTEXTE : Administration PKI, intégration des smartcards dans le système, gestion des certificats de confiance.

SOUS-COMMANDES :

list-modules	Liste les modules PKCS#11 configurés
list-tokens	Liste les tokens disponibles
list-objects	Liste les objets (certificats, clés...)
export-object	Exporte un objet
print-config	Affiche la configuration
server	Lance le mode serveur (proxy PKCS#11)

OPTIONS :

--module MODULE	Module PKCS#11 à utiliser
--token-label LABEL	Sélectionne le token par label
--verbose	Mode verbeux

EXEMPLES :

```
$ p11-kit list-modules
$ p11-kit list-tokens
$ p11-kit list-objects --token-label "MonToken"
$ p11-kit export-object --type cert "pkcs11:token=MonToken;object=MaCle"
```

```
[USER] trust (p11-kit trust)
```

DESCRIPTION : Gestion du trust store système via p11-kit – ajoute, supprime et liste les certificats de confiance du système

POURQUOI : Gérer les autorités de certification (CA) de confiance du système Linux de manière unifiée.

CONTEXTE : Administration PKI, ajout de CA d'entreprise, gestion des certificats racine.

SOUS-COMMANDES :

list	Liste les certificats de confiance
anchor FICHER	Ajoute un certificat comme ancre de confiance
dump	Affiche tous les objets du trust store

EXEMPLES :

```
$ trust list
$ sudo trust anchor --store /etc/pki/ca-trust/source/anchors/ mon_ca.crt
$ sudo update-ca-trust extract
```



```
$ trust dump | grep "label:"
```

```
[USER] tpm2_* / tss2_*
```

DESCRIPTION : Suites d'outils pour le TPM 2.0 (Trusted Platform Module) – puce de sécurité matérielle intégrée aux cartes mères modernes

POURQUOI : Le TPM est un HSM soudé sur la carte mère – il stocke des clés, mesure l'intégrité du système (PCR), protège les secrets au démarrage (BitLocker, LUKS via clevis/tang, ssh-tpm-agent...).

tpm2-tools : outils de bas niveau

tss2 (tpm2-tss-engine) : interface haut niveau FAPI

CONTEXTE : Sécurité du boot, chiffrement de disque, attestation, génération de nombres aléatoires hardware, stockage sécurisé de clés.

COMMANDES tpm2_* COURANTES :

tpm2_getrandom N	Génère N octets aléatoires via le TPM
tpm2_pcrread	Lit les registres PCR (Platform Configuration Registers)
tpm2_pcrextend	Étend un registre PCR
tpm2_createprimary	Crée une clé primaire dans une hiérarchie
tpm2_create	Crée une paire de clés dans le TPM
tpm2_load	Charge une clé dans le TPM
tpm2_sign	Signe des données avec une clé TPM
tpm2_verifysignature	Vérifie une signature TPM
tpm2_rsaencrypt	Chiffre avec RSA via le TPM
tpm2_rsadecrypt	Déchiffre avec RSA via le TPM
tpm2_nvread	Lit des données dans la NV RAM du TPM
tpm2_nvwrite	Écrit des données dans la NV RAM du TPM
tpm2_nvdefine	Définit un espace NV
tpm2_getcap	Affiche les capacités du TPM
tpm2_selftest	Lance l'auto-test du TPM
tpm2_clear	Efface le TPM (DANGER – perd toutes les clés)
tpm2_startup	Démarre le TPM
tpm2_shutdown	Arrête proprement le TPM
tpm2_hash	Calcule un hash via le TPM
tpm2_quote	Génère une attestation signée des PCR
tpm2_checkquote	Vérifie une attestation

COMMANDES tss2_* COURANTES (FAPI haut niveau) :

tss2_getrandom	Génère des octets aléatoires
tss2_createkey	Crée une clé via FAPI
tss2_sign	Signe via FAPI
tss2_verify / tss2_verifysignature	Vérifie une signature
tss2_encrypt / tss2_decrypt	Chiffre/déchiffre via FAPI
tss2_pcrread	Lit les PCR via FAPI
tss2_provision	Provisionne le TPM (init FAPI)
tss2_list	Liste les clés et objets FAPI

EXEMPLES :

```
$ tpm2_getrandom 32 | xxd
$ tpm2_pcrread sha256:0,1,2,3,4,5,6,7
$ tpm2_getcap properties-fixed
$ tpm2_selftest --fulltest
$ tpm2_createprimary -C o -g sha256 -G rsa -c primary.ctx
$ tpm2_create -C primary.ctx -g sha256 -G rsa -u key.pub -r key.priv
$ tpm2_load -C primary.ctx -u key.pub -r key.priv -c key.ctx
$ echo "data" | tpm2_sign -c key.ctx -g sha256 -o sig.rssa
$ tss2_provision
$ tss2_getrandom --size 32 --output random.bin
```